SU
SINAI UNIVERSITY
KANTARA

INT 453: Digital Multimedia

2024-2025

Lecturer : Dr. Maysa Khalil Noby
Dr. Reham Amin
Reham_Amin@ci.suez.edu.eg
Office: NG50

Compression Techniques

MPEG Moving Pictures Experts Group

- Started in 1988 as a working group within ISO/IEC
- Comprises a number of sub-groups e.g. Digital Storage Media (DSM), Delivery, Video, Audio
- Generates generic standards for digital video and audio compression
- MPEG-1: "Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to about 1.5 MBit/s"
- MPEG-2: "Generic Coding of Moving Pictures and Associated Audio"
- MPEG-3: no longer exists (was merged into MPEG-2)
- MPEG-4: “Coding of Audio-Visual Objects"
- MPEG-7: “Multimedia Content Description Interface”
- MPEG-21

Introduction

- Multimedia applications require essentially **Compression** techniques.
- Raw media takes much storage and bandwidth
  - A raw video with 30 frame/sec, resolution of 640x480, 24bit color
  - One second of video 30 * 640 * 480 * 3 = 27.6480 Mbytes

One hour video is about 100 Gbytes



What is compression?

Compression: the process of coding that will effectively reduce the total number of bits needed to represent certain information.

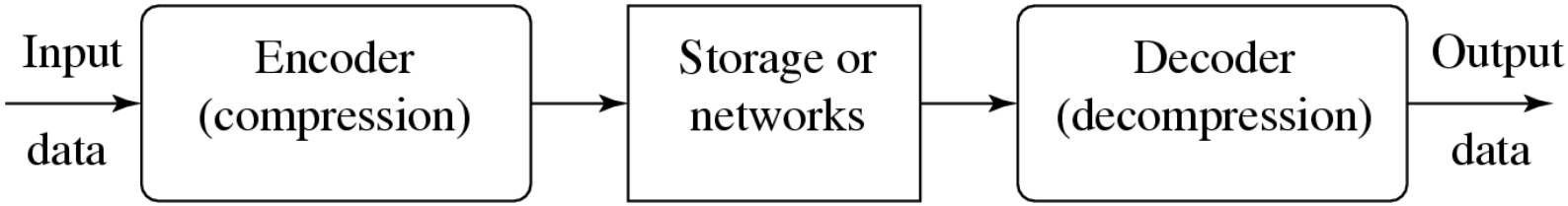


Figure. 1: A General Data Compression Scheme.

Why Data Compression?

- It can handle large amounts of multimedia data

- Data compression allows effectively faster transfer of information.
- Better usage can be made of lower speed connections hence improves network performance.
- Enhances security because it involves scrambling the data.



Data Compression Types

- Although many methods are used for this purpose, in general these methods can be divided into two broad categories: lossless and lossy methods.

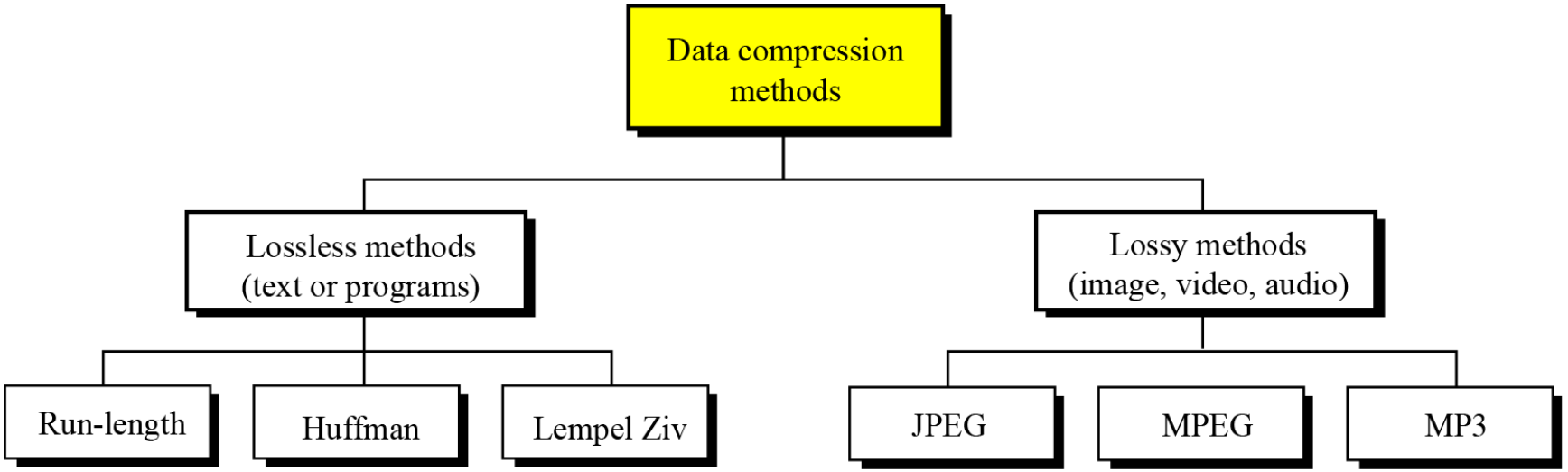


Figure 2: Data compression methods

Lossy versus Lossless

Lossy Compression:

Information is lost during compression

- Consumer images : web, Digital cameras

Lossless Compression:

Information preserving

- Medical imaging





Lossy Compression

Lossy compression reduces a file by permanently eliminating certain information, especially redundant information.

When the file is uncompressed, only a part of the original information is still there (although the user may not notice it).

Lossy compression is generally used for video and sound, where a certain amount of information loss will not be detected by most users



Lossless Compression

The integrity of the data is preserved.

- The original data and the data after compression and decompression are exactly the same.
- The compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process.
- Redundant data is removed in compression and added during decompression.
- Lossless compression methods are normally used when we cannot afford to lose any data.

Compression Ratio

$$compression\,ratio = \frac{B_0}{B_1} \quad (1)$$

B_0 – number of bits before compression

B_1 – number of bits after compression

Entropy Coding

The entropy express the average number of bits to encode a symbol.

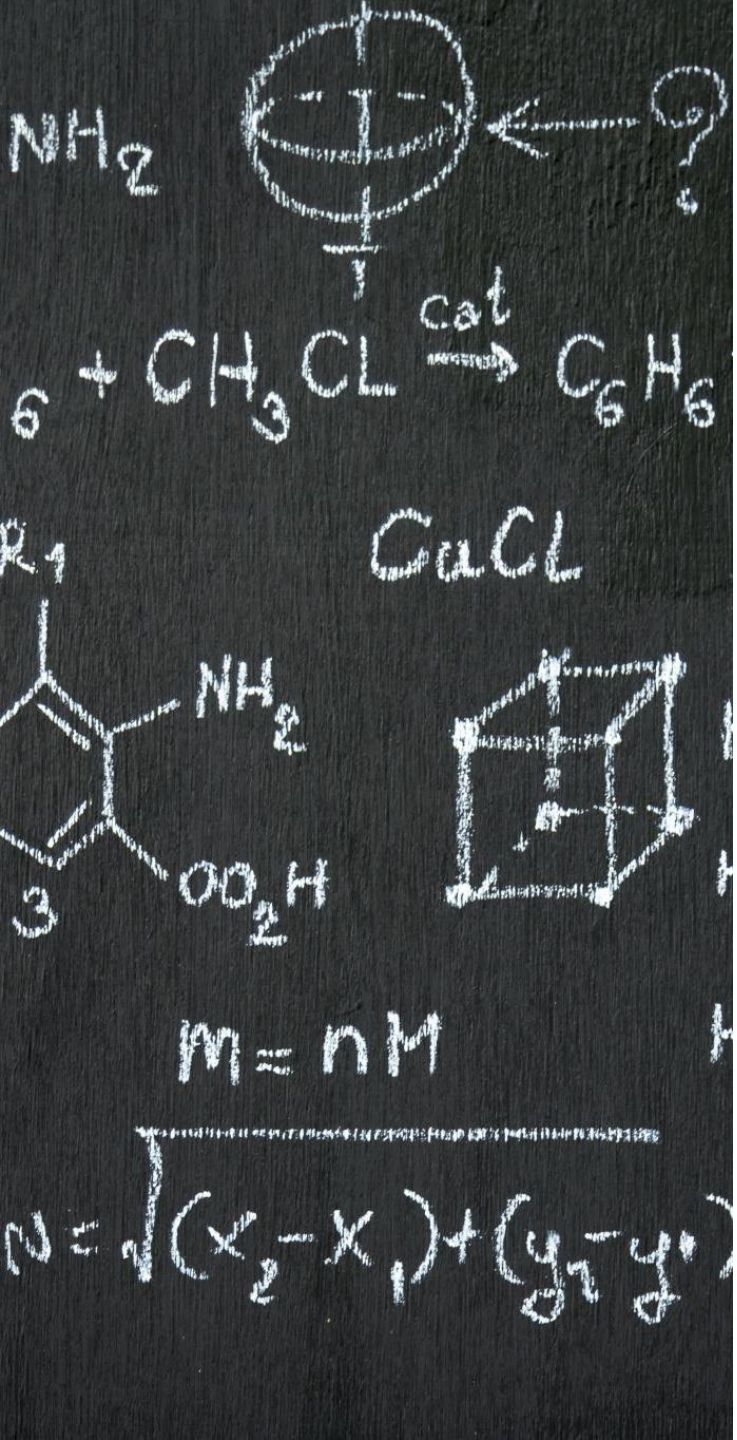
The entropy **η** of an information source with alphabet $S = \{s_1, s_2, \ldots, s_n\}$ is the average amount of information contained per symbol in the source S.

$$\eta = H(S) = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i} \quad (2)$$

$$= -\sum_{i=1}^{n} p_i \log_2 p_i \quad (3)$$

p_i – probability that symbol s_i will occur in S.

$\log_2 \frac{1}{p_i}$ – indicates the amount of information (self-information as defined by Shannon) contained in s_i, which corresponds to the number of bits needed to encode s_i.



Entropy Coding (cont.,)

Example:

A source outputs two symbols (the *alphabet* has 2 *letter*s) a or b.

P(a) = 0.25, P(b) = 0.75.

Entropy = H

$= 0.25 * \log_2 (1/0.25) + 0.75 * \log_2 (1/0.75)$

= 0.8113 bits

We need at least **0.8113** bits per symbol in **encoding**.

13

Entropy Coding Techniques

- Huffman Coding
- Run Length Coding
- Dictionary-based Coding (LZW)
- Arithmetic Coding



Run-Length Encoding

- RLE is probably the simplest method of compression.
- Content dependent coding
- RLE replaces the sequence of same consecutive bytes with the number of occurrences.
- It can be used to compress data made of any combination of symbols.
- The general idea behind this method is to replace consecutive repeating occurrences of a symbol by one occurrence of the symbol followed by the number of occurrences.
- The method can be even more efficient if the data uses only two symbols (for example 0 and 1) in its bit pattern and one symbol is more frequent than the other.

Run-length Encoding

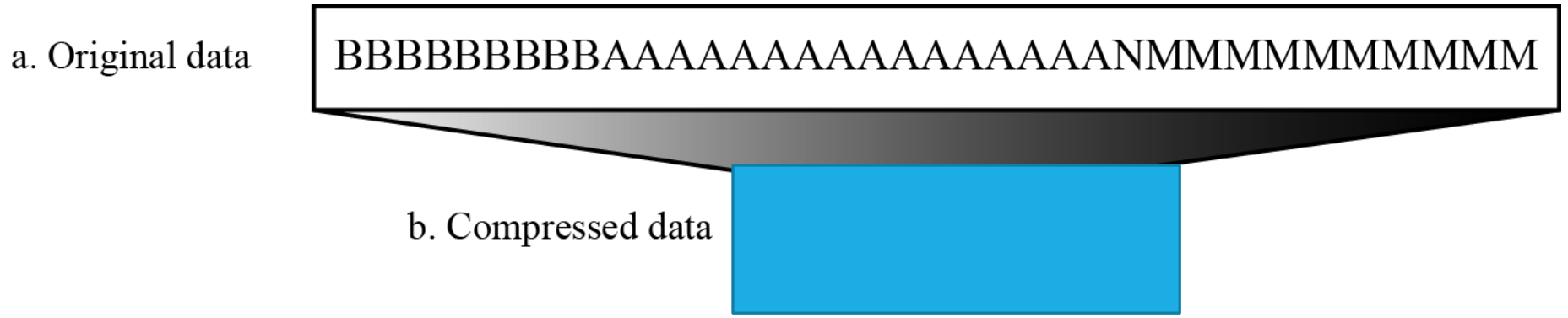


Figure 2. Run-length encoding example

Run-length Encoding

Example 2:

Input: “WWWWWWWWWBWWWWWWWWBBBWWWWWWWWWWB”

Output: “8WB7W3B10WB”

Run-length encoding for two symbols

- We can encode one symbol which is more frequent than the other.
- This example only encode 0's between 1's.

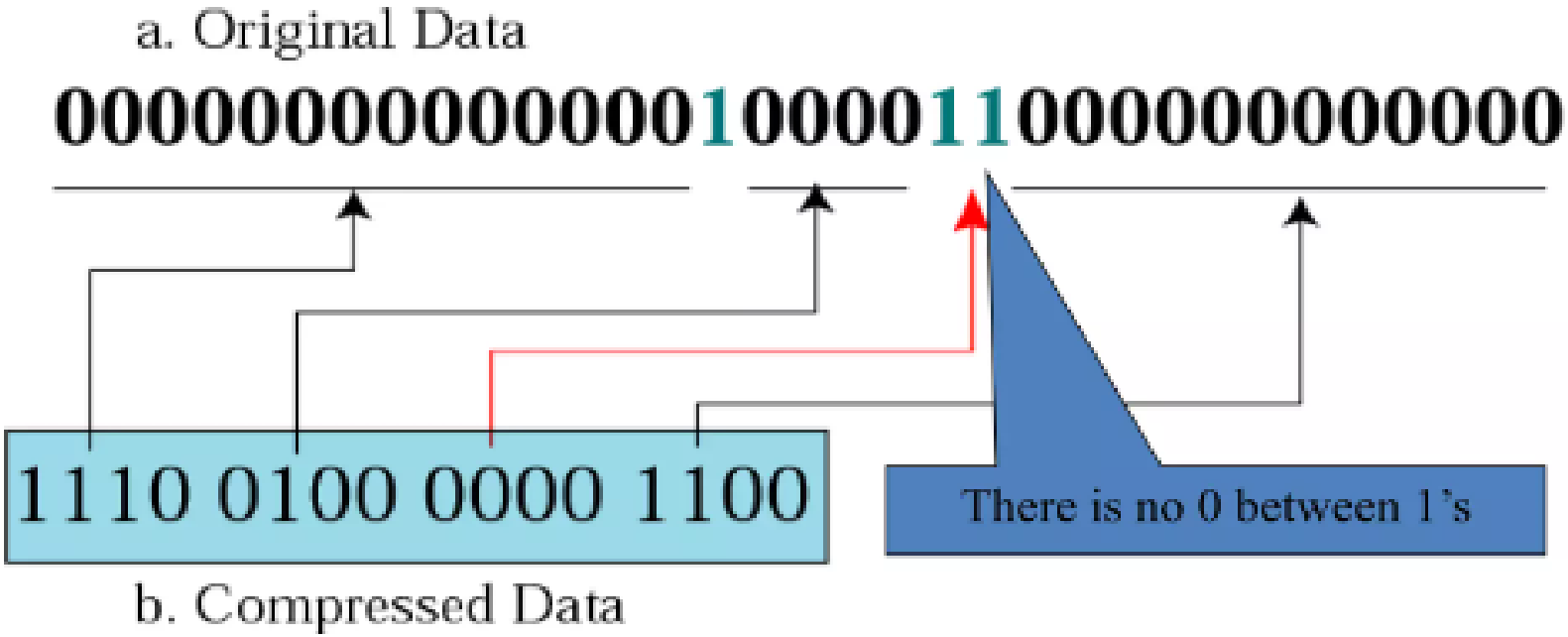


Figure 3. Run-length encoding for two symbols

Entropy Coding Techniques

Huffman Coding

Run Length Coding

Dictionary-based Coding (LZW)

Arithmetic Coding

Dictionary-based Coding



LZW uses fixed-length **codewords** to represent variable-length strings of symbols/characters that commonly occur together, e.g., words in English text.



The **LZW** encoder and decoder build up the same dictionary dynamically while receiving the data.



LZW places longer and longer repeated entries into a dictionary, and

Then emits the ***code*** for an element, rather than the string itself, if the element has already been placed in the dictionary.

ALGORITHM - LZW Compression

BEGIN

```
s = next input character;
while not EOF
{
        c = next input character;
        if s + c exists in the dictionary
                s = s + c;
        else
                {
        output the code for s;
        add string s + c to the dictionary with a
        new code;
        s = c;
                }
}
output the code for s;
```

END

ALGORITHM - LZW Compression

In summary

BEGIN

if s + c not in the dictionary:

Assign new code to (s+c)

Add (s + c) to dictionary with new code

Continue with C

Else:

No coding

Continue with S+C

output the code for s with the same range;

END

Example (2) LZW compression for string "ABABBABCABABBA"

Let's start with a very simple dictionary (also referred to as a "string table"), initially containing only 3 characters, with codes as follows:

Code	String
1	A
2	B
3	C

Apply the **LZW Compression algorithm** to **decompress** the following code

"ABABBABCABABBA"

Afterwards, **determine** the compression ratios.

ABABBABCABABBA

S	C(Next)	Output Code (for S)	New Code	String (S+C) (Dictionary)	
			1 2 3	A B C	
A	B	1	4 (new code)	AB	Not in dict
B	A	2	5	BA	Not in dict
A	B				in dict
AB	B	4	6	ABB	Not in dict
B	A				in dict
BA	B	5	7	BAB	Not in dict
B	C	2	8	BC	Not in dict
C	A	3	9	CA	Not in dict
A	B				in dict
AB	A	4	10	ABA	Not in dict
A	B				in dict
AB	B				in dict
ABB	A	6	11	ABBA	Not in dict
A	EOF	1			

- The output codes are: **1 2 4 5 2 3 4 6 1** (the same as Output Code (for S) in table). Instead of sending 14 characters, only 9 codes need to be sent (compression ratio = 14/9 = 1.56).

ABABBABCABABBA

S	C(Next)	Output (for S)	Code	String (S+C) (Dictionary)
			1 2 3	A B C
A B A AB B BA B C A AB A AB ABB A	B A B B A B C A B A B B A EOF	1 2 4 5 2 3 4 6 1	4 (new code) 5 6 7 8 9 10 11 	AB BA ABB BAB BC CA ABA ABBA

- The output codes are: 1 2 4 5 2 3 4 6 1. Instead of sending 14 characters, only 9 codes need to be sent (compression ratio = 14/9 = 1.56).

ALGORITHM: LZW Decompression (simple version)

```
BEGIN
  s = NIL;
  while not EOF
  {
     k = next input code;
     entry = dictionary entry for k;
     output entry;
     if (s != NIL)
           add string s + entry[0] to dictionary
  with a new code;
     s = entry;
  }
END
```

Example (3) LZW decompression for 1 2 4 5 2 3 4 6 1

- To decompress the encoded string
- Look at the table and replace each number (code) with the corresponding string in the dictionary
- Encoded: 1| 2| 4| 5| 2| 3| 4 |6 |1
- Decoded: A|B|AB|BA|B|C|AB|ABB|A|

Code	String (S+C) (Dictionary)
1	A
2	B
3	C
4 (new code)	AB
5	BA
6	ABB
7	BAB
8	BC
9	CA
10	ABA
11	ABBA

Apparently, the output string is “ABABBABCABABBA”, a truly lossless result!

ALGORITHM : LZW Decompression (modified)

```
BEGIN
 s = NIL;
 while not EOF
 {
  k = next input code;
  entry = dictionary entry for k;
  /* exception handler */
  if (entry == NULL)
      entry = s + s[0];
  output entry;
  if (s != NIL)
      add string s + entry[0] to dictionary with a new
  code;
  s = entry;
 }

END
```

The initial codes for letters A, B and C are 1, 2 and 3, respectively.

Apply the **LZW Compression algorithm** to **decompress** the following code

1 4 5 4 2 8 9 3 11 12 13 9 9 6 6

Afterwards, **determine** the compression ratios.

S	K(next input code)	Entry/output (Dic. Entry at k)	Output Code (new code to output Dic.)	String (To output Dic.) S + Entry[0]
			1	A
			2	B
			3	C
Nil	1	A		
A	4	AA	4	AA
AA	5	AAA	5	AAA
AAA	4	AA	6	AAAA
AA	2	B	7	AAB
B	8	BB	8	BB
BB	9	BBB	9	BBB
BBB	3	C	10	BBBC
C	11	CC	11	CC
CC	12	CCC	12	CCC
CCC	13	CCCC	13	CCCC
CCCC	9	BBB	14	CCCCB
BBB	9	BBB	15	BBBB
BBB	6	AAAA	16	BBBA
AAAA	6	AAAA	17	AAAAA

AAAAAAAABBBBBBCCCCCCCCCBBBBBBAAAAAAAA

Compression Ratio = 38/15 = 2.53

Complete LZW coding for Images

1. Similar to text
2. Replace the method that reads a text file by a method reads a binary file.
3. Replace the method that writes a text file by a method writes a binary file.

Assignment

What is data redundancy types?

Compare between Huffman Coding and Run length Coding?

Questions

Thank you